

COIMBATORE INSTITUTE OF TECHNOLOGY
Civil Aerodrome Post, Coimbatore,
Tamilnadu, India – 641014



Python Programming Laboratory (22MDC27)

STUDENT EXPENSE TRACKER

Submitted By :

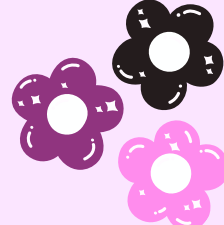
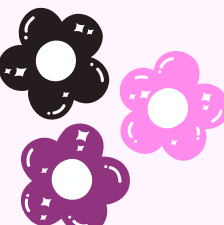
Meghaa Sree. S
(2403717673722026)

Mithra Omprakash
(2403717673722027)

Submitted To:

Dr.V. Savithri
Dr.S. Chandia





ABSTRACT

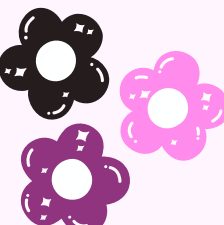
PROBLEM STATEMENT:

Managing personal finances is a common challenge faced by students, especially when it comes to tracking daily expenses and understanding spending habits. Without a clear overview of where their money goes, students often struggle with budgeting, which can lead to overspending or insufficient savings.

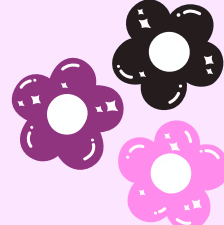
To address this issue, this project proposes the development of a Flask-based Student Expense Tracker Web Application. The system enables students to record their expenses and visualize their spending patterns through interactive and insightful graphs. It uses a CSV file as the data source and presents analyses such as:

- Overall spending statistics
- Category-wise expense distribution
- Gender-based and age-group-based comparisons
- Monthly expense trends
- Top spenders overview

The application also includes a form to add new expenses and a feature to download the expense report. By offering clear and visual financial insights, the tracker empowers students to make informed decisions about their spending and improve their financial habits.



ABSTRACT



PROPOSED SOLUTION:

To help students effectively manage and analyze their expenses, the proposed solution is a web-based Student Expense Tracker developed using Flask (Python web framework). The solution is designed to be simple, interactive, and informative, making it easy for students to track and understand their financial habits. The key features of the proposed system are:

◆ 1. User-Friendly Web Interface

- A clean and intuitive interface built with HTML templates.
- Easy navigation between home, analytics, and data entry pages.

◆ 2. Expense Data Management

- Uses a CSV file as a lightweight data source to store and retrieve expense records.
- Allows users to add new expense entries via a form (with fields like student ID, gender, age, category, amount, and date).

◆ 3. Visual Expense Analysis

Generates multiple charts using Matplotlib to offer deep insights into student expenses:

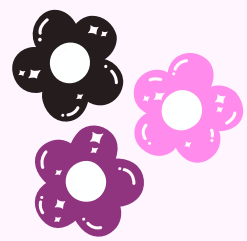
- Overall Statistics: Total number of students, total and average expenses, gender-wise totals.
- Category-wise Analysis: Bar charts showing expense distribution across categories.
- Gender-Based Analysis: Pie charts and comparative bar charts for male vs. female spending.
- Age Group Analysis: Bar charts showing average spending across different age groups.
- Monthly Trends: Line charts showing monthly expense patterns by gender.

◆ 4. Top Spenders Section

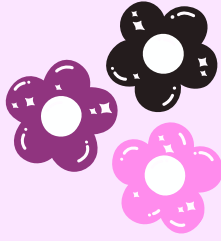
- Identifies and displays students with the highest total expenses to highlight significant spenders.

◆ 5. Report Download

- Offers a simple button to download the entire dataset as a CSV report for offline use or further analysis.



TECHNOLOGIES AND TOOLS



PYTHON:

- Python is a beginner-friendly programming language used to write the main logic of the project.
- It handles the data, processes the form input, and runs the web server.

FLASK FRAMEWORK:

- Flask is a lightweight tool that lets you build websites using Python.
- It helps connect your Python code to a web page so users can interact with it.

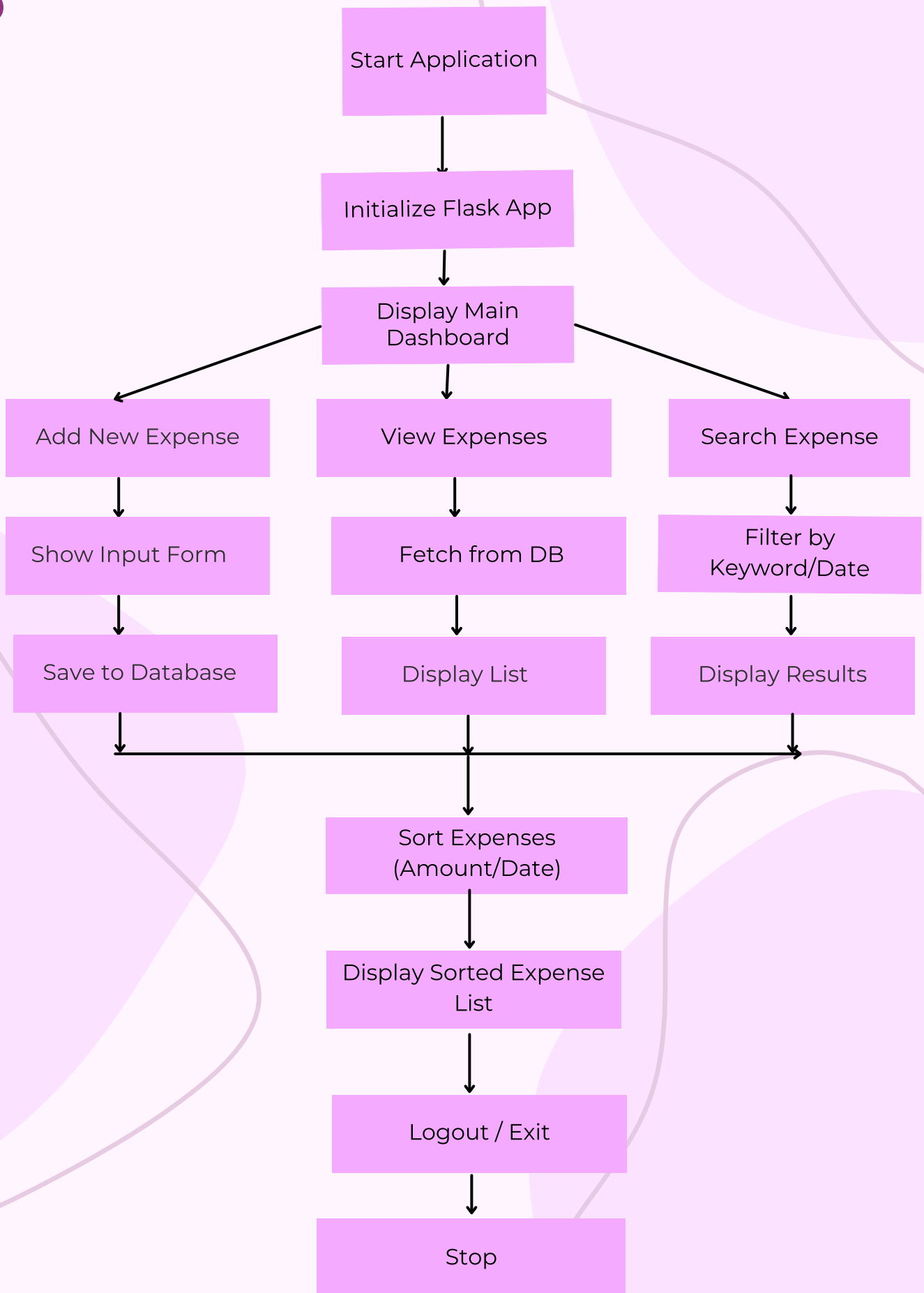
HTML:

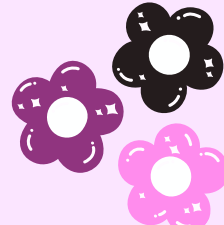
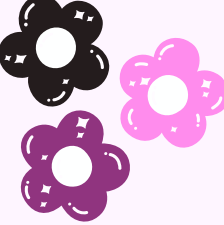
- HTML is used to create the structure of your webpage, like headings, buttons, and input fields.
- It's what you see on the screen when you open the website in a browser.

CSS:

- CSS adds style to your webpage, like colors, spacing, and fonts.
- It makes the website look nicer, but the app can work even without it.

SYSTEM FLOW DIAGRAM





ALGORITHM

Step 1: Initialize Application

- 1.Import necessary modules: Flask, pandas, matplotlib.pyplot, datetime, os, etc.
- 2.Define Flask app and set secret key for flashing messages.
- 3.Set the dataset path (CSV file).
- 4.Check if the CSV exists:
 - If yes, load it into a DataFrame.
 - If no, initialize an empty DataFrame.
- 5.Convert Date column to datetime format and remove invalid entries.
- 6.Ensure static/images directory exists for storing charts.

Step 2: Home Page Route

- 1.Define / route.
- 2.Render the home.html template with navigation options.

Step 3: Overall Statistics Route

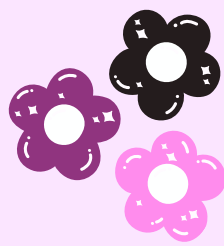
- 1.Define /overall route.
- 2.Calculate:
 - Total unique students.
 - Total expense.
 - Average expense.
 - Total male expense.
 - Total female expense.
- 3.Render overall.html with calculated values.

Step 4: Gender Analysis Route

- Define /gender-analysis route.
- Convert Age column to numeric and drop invalid values.
- Generate and save the following charts:
 - Bar chart: Category-wise expense by gender.
 - Pie charts: Category distribution for Male and Female.
 - Bar chart: Average expense by age group and gender.
 - Bar chart: Total expense by gender.
 - Line chart: Monthly expense trends by gender.
- Render gender_analysis.html with generated visualizations.



ALGORITHM



Step 5: Category Expense Trends Route

1. Define /category_expense_trends route.
2. Group data by category and calculate total amount.
3. Generate and save a bar chart.
4. Render category_expense_trends.html.

Step 6: Top Spenders Route

1. Define /top-spenders route.
2. Group by StudentID and calculate total expenses.
3. Sort and get top 5 spenders.
4. Render top_spenders.html with the list.

Step 7: Add Expense Route

1. Define /add-expense route with GET and POST methods.
2. If method is POST:
 - Get form values: StudentID, Gender, Age, Category, Amount, Date.
 - Create a new DataFrame row.
 - Append to the main DataFrame.
 - Save the updated DataFrame to the CSV file.
 - Flash a success message and redirect to the same page.
3. If method is GET, render the add_expense.html form page.

Step 8: Download Report Route

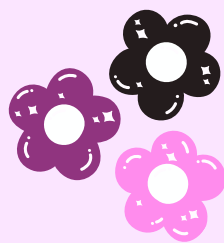
1. Define /download-report route.
2. Check if the CSV file exists.
 - If yes, send it as a downloadable file.
 - If no, flash an error message and redirect to the home page.

Step 9: Run the Flask Server

1. Run the app with debug=True for development purposes.



SOURCE CODE



```
from flask import Flask, render_template, request,
redirect, url_for, flash, send_file
import pandas as pd
import os
import tempfile
import shutil
import matplotlib
matplotlib.use('Agg') # Use non-interactive backend
to avoid GUI errors
import matplotlib.pyplot as plt
from datetime import datetime

app = Flask(__name__)
app.secret_key = 'your_secret_key'

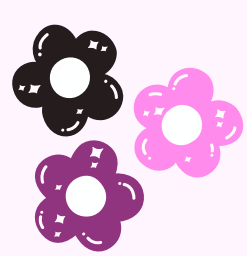
# Set dataset path
DATA_PATH =
r"C:\Users\HP\OneDrive\Desktop\student_expense_tracker\dataset.csv"

# Ensure image directory exists
os.makedirs("static/images", exist_ok=True)

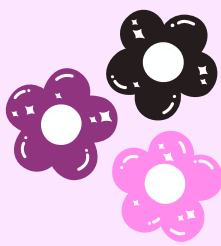
# Function to safely load the dataset
def load_data():
    if os.path.exists(DATA_PATH):
        df = pd.read_csv(DATA_PATH)
        df['Date'] = pd.to_datetime(df['Date'],
errors='coerce')
        df = df.dropna(subset=['Date'])
        df['Age'] = pd.to_numeric(df['Age'],
errors='coerce')
        df = df.dropna(subset=['Age'])

        df['Amount'] = pd.to_numeric(df['Amount'],
errors='coerce')
        df = df.dropna(subset=['Amount'])

        df['Category'] = df['Category'].str.strip().str.title()
    return df
return pd.DataFrame()
```

SOURCE CODE



```
@app.route('/')
def home():
    return render_template("home.html")

@app.route('/overall')
def overall():
    df = load_data()
    if df.empty:
        flash("No data available.", 'danger')
        return redirect(url_for('home'))

    return render_template("overall.html",
        total_students=df['StudentID'].nunique(),
        total_expense=df['Amount'].sum(),
        avg_expense=round(df['Amount'].mean(), 2),
        male_expense=df[df['Gender'] == 'Male']
        ['Amount'].sum(),
        female_expense=df[df['Gender'] == 'Female']
        ['Amount'].sum())

@app.route('/gender-analysis')
def gender_analysis():
    df = load_data()
    if df.empty:
        flash("No data available for analysis.", 'danger')
        return redirect(url_for('home'))

    # Category-wise bar chart
    category_expense = df.groupby(['Gender',
    'Category'])['Amount'].sum().unstack().fillna(0)
    category_expense.T.plot(kind='bar', figsize=(10, 6))
    plt.title('Category-wise Expense by Gender')
    plt.ylabel('Total Expense')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig("static/images/category_bar.png")
    plt.close()
```

SOURCE CODE

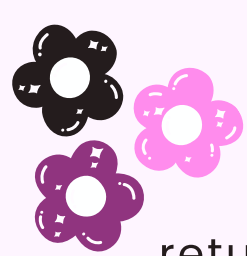
```
# Pie charts
for gender in ['Male', 'Female']:
    cat_data = df[df['Gender'] ==
gender].groupby('Category')['Amount'].sum()
    plt.figure(figsize=(6, 6))
    plt.pie(cat_data, labels=cat_data.index,
autopct='%1.1f%%', startangle=140)
    plt.title(f'{gender} Expense Distribution')
    plt.tight_layout()

plt.savefig(f"static/images/{gender.lower()}_pie.png")
plt.close()

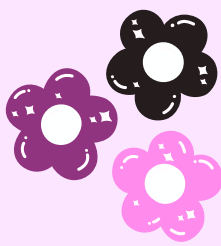
# Age group vs avg expense
df['AgeGroup'] = pd.cut(df['Age'], bins=[17, 20, 23, 26,
29], labels=["18-20", "21-23", "24-26", "27-29"])
age_avg_expense = df.groupby(['AgeGroup',
'Gender'])['Amount'].mean().unstack().fillna(0)
age_avg_expense.plot(kind='bar', figsize=(10, 6))
plt.title('Avg Expense by Age Group and Gender')
plt.ylabel('Average Expense')
plt.tight_layout()
plt.savefig("static/images/age_group_bar.png")
plt.close()

# Gender total bar
gender_total = df.groupby('Gender')['Amount'].sum()
gender_total.plot(kind='bar', color=['skyblue',
'lightpink'])
plt.title('Total Expense by Gender')
plt.ylabel('Total Expense')
plt.tight_layout()
plt.savefig("static/images/gender_total.png")
plt.close()

# Monthly trend
df['Month'] = df['Date'].dt.to_period('M')
monthly = df.groupby(['Month', 'Gender'])
['Amount'].sum().unstack().fillna(0)
monthly.plot(figsize=(10, 6))
plt.title('Monthly Expense Trend by Gender')
plt.ylabel('Total Expense')
plt.xticks(rotation=45)
plt.tight_layout()
plt.savefig("static/images/monthly_trend.png")
plt.close()
```



SOURCE CODE



```
return render_template("gender_analysis.html")

@app.route('/category_expense_trends')
def category_expense_trends():
    df = load_data()
    if df.empty:
        flash("No data available for trend analysis.", 'danger')
        return redirect(url_for('home'))

    category_expense_data = df.groupby('Category')
    ['Amount'].sum().sort_values(ascending=False)
    plt.figure(figsize=(10, 6))
    category_expense_data.plot(kind='bar',
    color='mediumseagreen')
    plt.title('Category-wise Expense Trends')
    plt.xlabel('Category')
    plt.ylabel('Total Expense')
    plt.xticks(rotation=45)
    plt.tight_layout()
    plt.savefig('static/category_expense_trend.png')
    plt.close()

    return render_template('cat_ex.html')

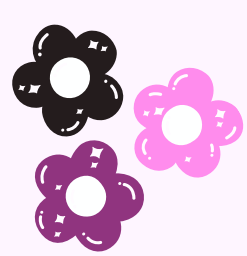
@app.route('/top-spenders')
def top_spenders():
    # Read dataset
    df = pd.read_csv('dataset.csv')

    # Group by StudentID and sum Amount
    grouped = df.groupby('StudentID')
    ['Amount'].sum().reset_index()

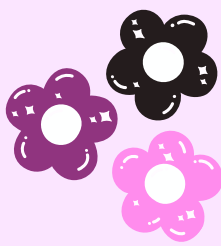
    # Sort in descending order and get top 5
    top_spenders_list =
    grouped.sort_values(by='Amount',
    ascending=False).head(5).values.tolist()

    return render_template('meee.html',
    top_spenders=top_spenders_list)

@app.route('/add-expense', methods=['GET', 'POST'])
def add_expense():
    if request.method == 'POST':
        student_id = request.form['student_id']
```



SOURCE CODE



```
gender = request.form['gender']
age = request.form['age']
category = request.form['category']
amount = request.form['amount']
date = request.form['date']

try:
    new_entry = pd.DataFrame({
        'StudentID': [student_id],
        'Gender': [gender],
        'Age': [float(age)],
        'Category': [category.strip().title()], # Normalize new
entry category
        'Amount': [float(amount)],
        'Date': [pd.to_datetime(date)]
    })

    df = load_data()
    df = pd.concat([df, new_entry], ignore_index=True)

    # Save using a temp file to avoid overwrite errors
    with tempfile.NamedTemporaryFile(mode='w',
delete=False, suffix='.csv') as tmp:
        df.to_csv(tmp.name, index=False)
        shutil.move(tmp.name, DATA_PATH)

    flash("Expense added successfully!", 'success')
except Exception as e:
    flash(f"Error adding expense: {e}", 'danger')

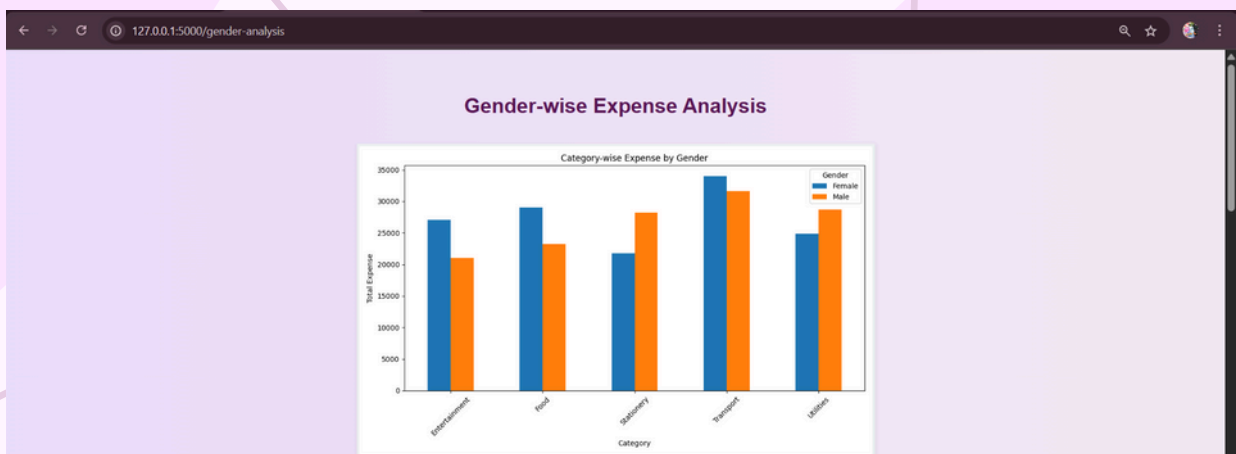
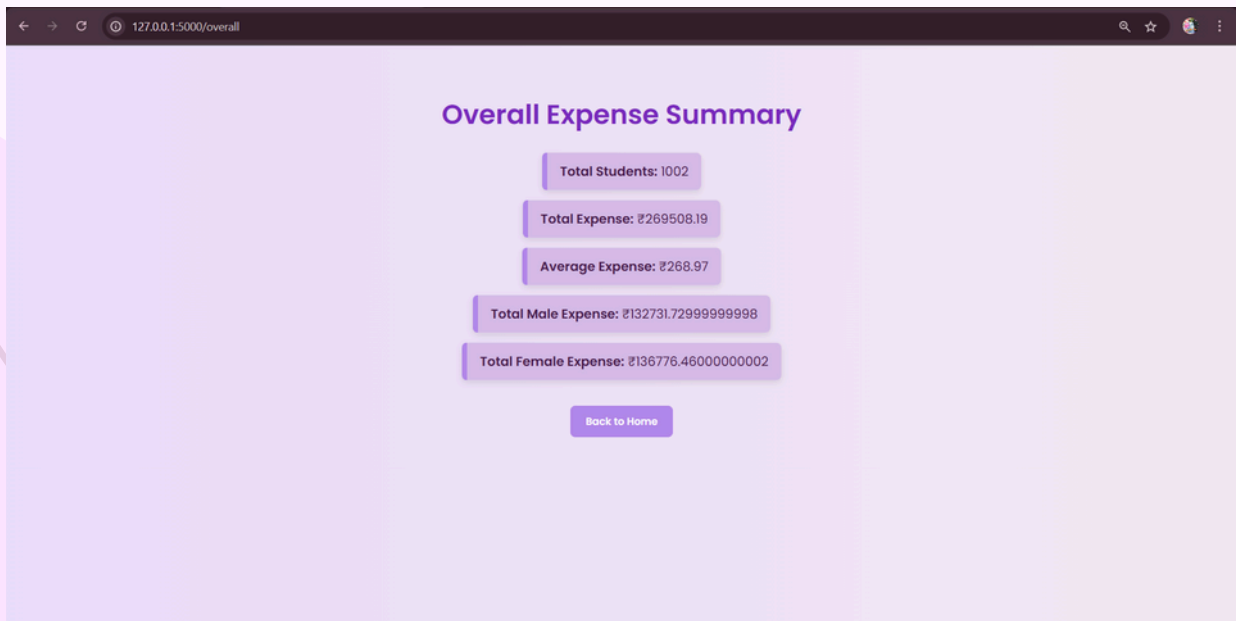
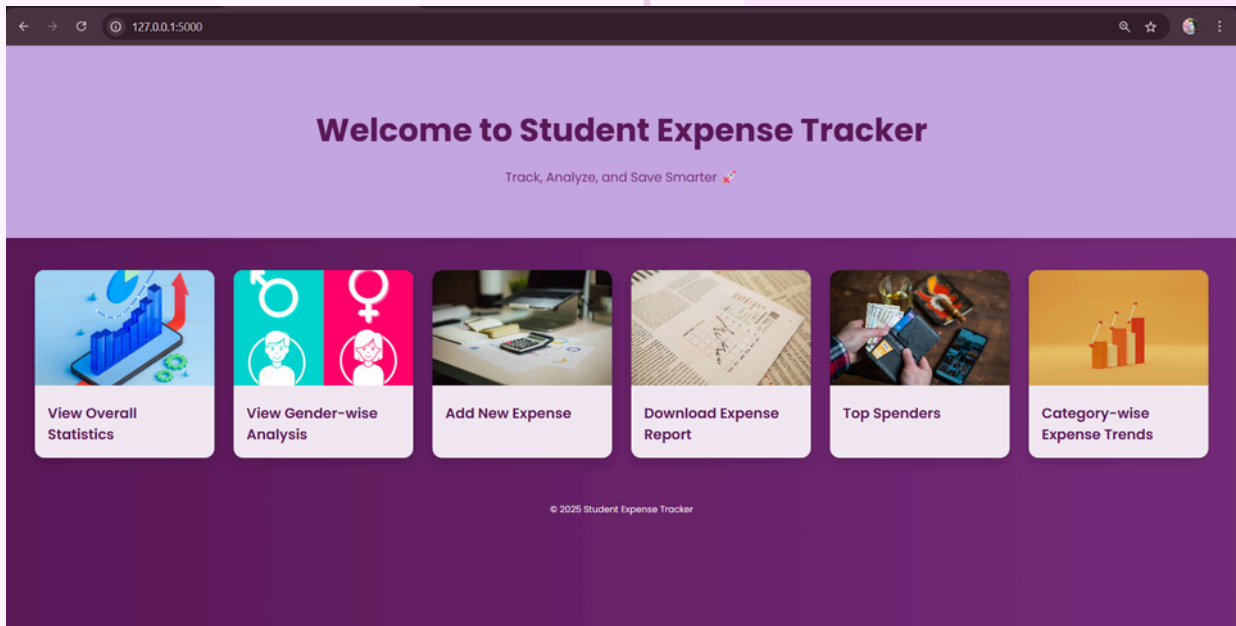
return redirect(url_for('add_expense'))

return render_template("add_expense.html")

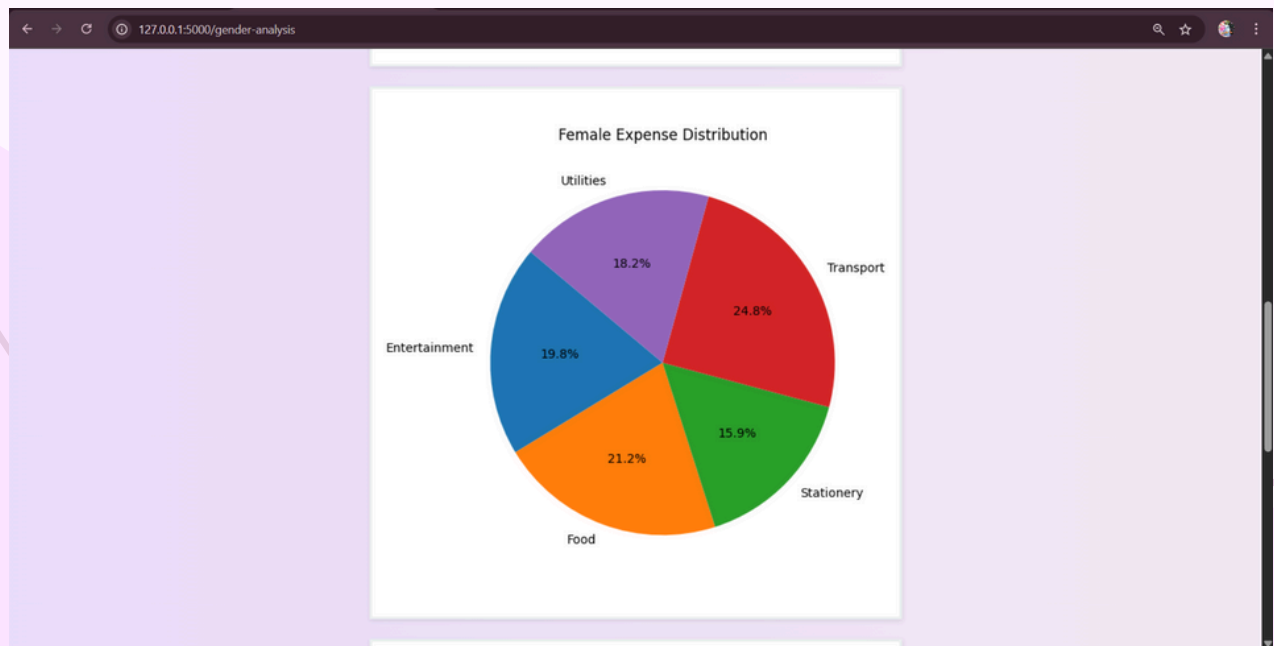
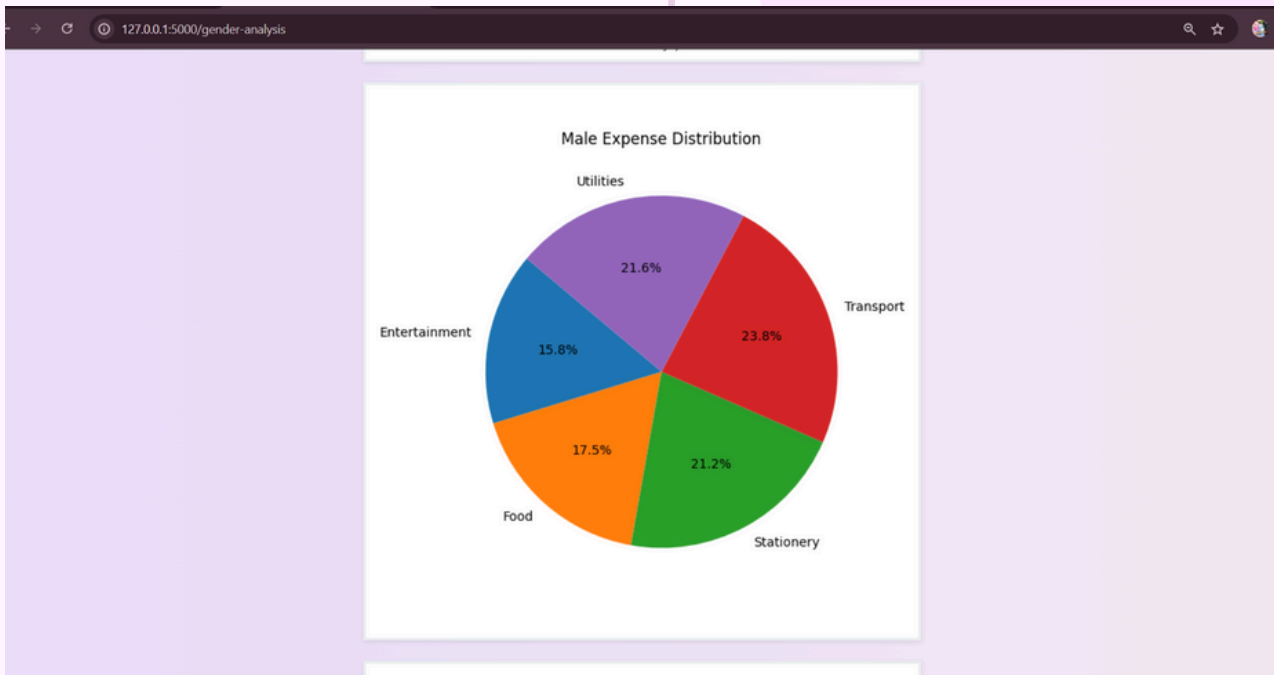
@app.route('/download-report')
def download_report():
    if os.path.exists(DATA_PATH):
        return send_file(DATA_PATH, as_attachment=True)
    else:
        flash("No report available for download.", 'danger')
        return redirect(url_for('home'))

if __name__ == '__main__':
    app.run(debug=True)
```

OUTPUT



OUTPUT



OUTPUT

127.0.0.1:5000/add-expense

Add New Expense Entry

-- Select Gender --

Add Expense

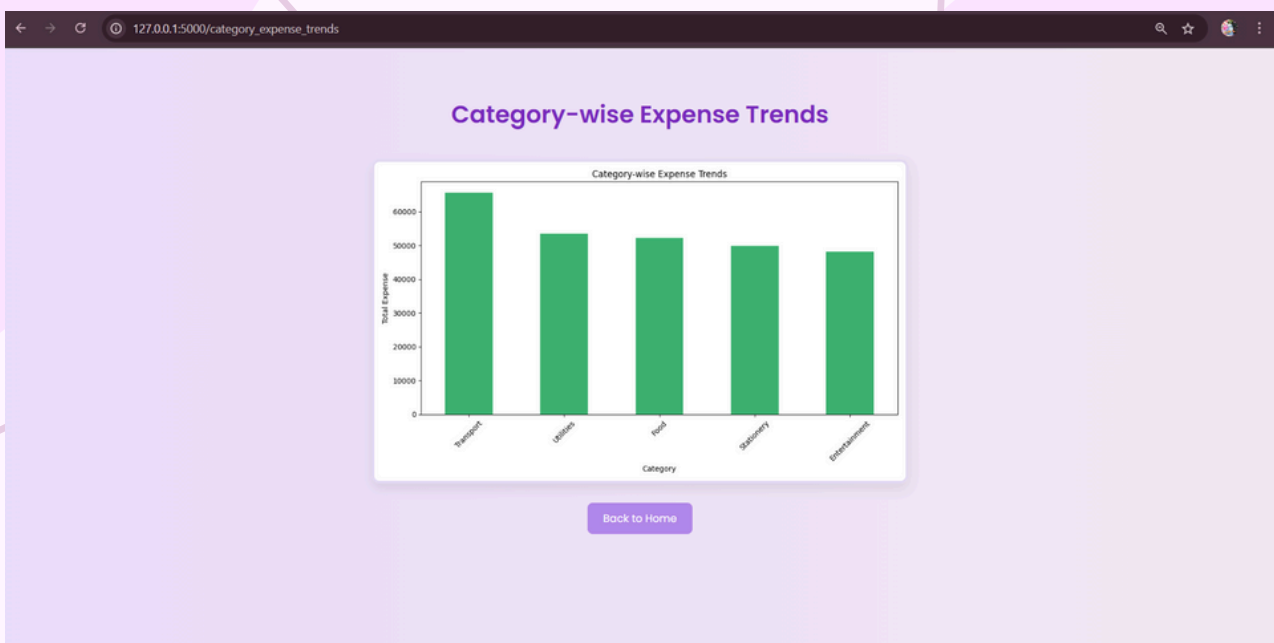
Back to Home

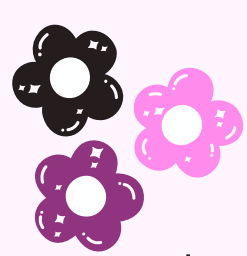
127.0.0.1:5000/top-spenders

Top 5 Spenders

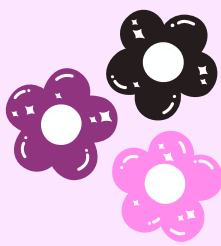
Student ID	Total Amount
8902	499.89
3343	499.85
8202	499.62
3346	497.66
5515	497.36

Back to Home



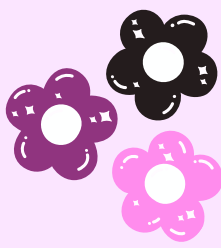
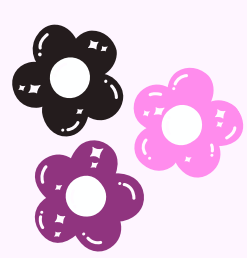


RESULT ANALYSIS



The Student Expense Tracker Web Application was successfully developed and tested using Python (Flask framework), Pandas for data manipulation, and Matplotlib for data visualization. The system achieved the following outcomes:

- ✓ 1. Effective Data Entry & Storage
 - Students were able to add their expense details via a simple web form.
 - Data entries were saved persistently to a CSV file (dataset.csv), enabling future retrieval and analysis.
 - Each record stored: Student ID, Gender, Age, Category, Amount, and Date.
- ✓ 2. Overall Expense Insights
 - Total number of unique student users was calculated.
 - Total and average expenses were derived and presented.
 - Gender-based total expense comparison (Male vs. Female) was displayed.
- ✓ 3. Gender-wise Analysis with Visuals
 - Category-wise bar chart by gender helped compare spending habits across different needs.
 - Pie charts showed proportional spending for each gender.
 - Age group vs. average expense chart gave insights into spending behaviors across age brackets.
 - Monthly trends showed how spending changes over time, split by gender.
- ✓ 4. Category-Wise Trends
 - Categories such as Food, Travel, Stationery, etc., were analyzed to show where students spent most.
 - A clear bar graph identified the most expensive categories.
- ✓ 5. Top Spenders
 - The top 5 students with the highest total expenses were identified and displayed, motivating awareness and budget control.
- ✓ 6. Reporting
 - The application supported downloading of the complete dataset as a report (.csv), enabling further offline analysis or record-keeping.

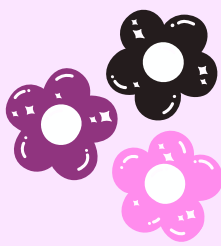
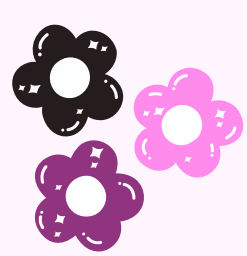


INFERENCE

The developed Student Expense Tracker system effectively demonstrates how technology can simplify personal finance management for students. Through the use of data collection, analysis, and visualization:

- Students gain clear visibility into their spending patterns across various categories like food, travel, and stationery.
- The system provides gender-wise and age-based insights, helping identify specific trends and behaviors.
- Monthly trend analysis enables students to observe changes over time, encouraging better budgeting and self-discipline.
- The application interface is simple, interactive, and informative, making it accessible even for non-technical users.

In conclusion, this project validates that even a lightweight web solution, when powered by proper data analytics, can empower students to make smarter financial decisions and improve their financial awareness and responsibility.



FUTURE ENHANCEMENTS

1. User Authentication and Profiles

- Enable secure login/signup for students.
- Each student can view and manage only their own expenses.

2. Dashboard with Filters

- Add interactive filters for date range, categories, or monthly views.
- Allow students to visualize their specific data dynamically.

3. Mobile-Friendly Interface

- Make the UI fully responsive for mobile and tablet users using frameworks like Bootstrap or Tailwind CSS.

4. Budget Setting and Alerts

- Let users set monthly budget limits.
- Notify them with warnings or emails when nearing/exceeding the limit.

5. Expense Prediction Using Machine Learning

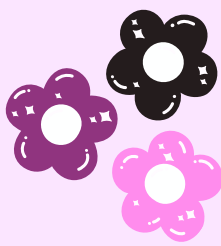
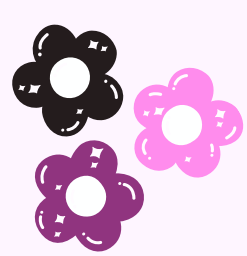
- Predict future expenses based on past patterns.
- Help students plan finances ahead of time.

6. Data Export in Multiple Formats

- Allow users to download their expense reports in PDF, Excel, or image format.

7. Recurring Expense Tracking

- Support automatic logging of recurring expenses like rent, subscriptions, etc.



REFERENCES

- 1.Flask Web Framework
 - Flask Documentation:
<https://flask.palletsprojects.com/>
- 2.Pandas for Data Handling
 - Pandas Documentation:
<https://pandas.pydata.org/docs/>
- 3.Matplotlib for Data Visualization
 - Matplotlib Documentation:
<https://matplotlib.org/stable/contents.html>
- 4.Python Standard Library
 - Official Python Docs:
<https://docs.python.org/3/>
- 5.HTML & Jinja2 Templating
 - Jinja2 Template Engine:
<https://jinja.palletsprojects.com/>
- 6.W3Schools HTML Guide:
 - <https://www.w3schools.com/html/>
- 7.CSV File Format Reference
 - RFC 4180 (CSV Standard):
<https://tools.ietf.org/html/rfc4180>
- 8.Visual Design Ideas & UI Patterns
- 9.Pinterest for inspiration on layout and charts.